

# Java-React Hybrid Chat Application

Design, Architecture, and Implementation Details

**Submitted by:** Akash Kundu

December 8, 2025

# Contents

|          |                                     |          |
|----------|-------------------------------------|----------|
| <b>1</b> | <b>Project Overview</b>             | <b>3</b> |
| 1.1      | Concept . . . . .                   | 3        |
| 1.2      | System Description . . . . .        | 3        |
| <b>2</b> | <b>Project Directory Structure</b>  | <b>3</b> |
| <b>3</b> | <b>Technology Stack &amp; Tools</b> | <b>4</b> |
| 3.1      | Frontend (The Client) . . . . .     | 4        |
| 3.2      | Middleware (The Bridge) . . . . .   | 4        |
| 3.3      | Backend (The Server) . . . . .      | 4        |
| 3.4      | Database . . . . .                  | 4        |
| <b>4</b> | <b>Implementation Details</b>       | <b>4</b> |
| 4.1      | Backend Implementation . . . . .    | 4        |
| 4.2      | Bridge Implementation . . . . .     | 5        |
| 4.3      | Frontend Implementation . . . . .   | 5        |
| <b>5</b> | <b>UML Architecture Diagram</b>     | <b>5</b> |

# 1 Project Overview

## 1.1 Concept

This project is a decoupled real-time messaging system. It demonstrates a sophisticated "Hybrid" architecture where a modern web frontend (React) communicates with a traditional, high-performance threaded backend (Java) via a lightweight middleware bridge (Node.js).

The core philosophy is **Separation of Concerns**:

- **Frontend**: Handles UI/UX and WebSocket connections.
- **Bridge**: Handles protocol translation (WebSocket ↔ TCP).
- **Backend**: Handles business logic, threading, and database persistence.

## 1.2 System Description

The application supports persistent user authentication, real-time private messaging, and history retrieval. It solves the problem of connecting browser-based clients (which speak WebSocket) to raw TCP socket servers using a Node.js intermediary.

# 2 Project Directory Structure

The project is organized into three distinct modules.

```
/ (Root Project)
├── 1_backend_java (Core Server)
│   ├── src/com/chat
│   │   ├── Main.java
│   │   ├── core/ConnectionListener.java
│   │   ├── services
│   │   │   ├── AuthManager.java
│   │   │   ├── ClientManager.java
│   │   │   └── DatabaseManager.java
│   │   ├── threads/ClientWorker.java
│   │   ├── models (User, Message)
│   │   └── utils (Logger, PasswordHasher)
│   └── lib/mongo-java-driver.jar
├── 2_bridge_node (Middleware)
│   ├── src
│   │   ├── main.js
│   │   ├── services
│   │   │   ├── tcpService.js
│   │   │   └── websocketService.js
│   │   └── config/appConfig.js
│   └── package.json
└── 3_client_web (Frontend)
    ├── src
    │   ├── App.jsx
    │   ├── main.jsx
    │   ├── components/ChatMinimal.jsx
    │   └── hooks/useChatBridge.js
```

```
├─ vite.config.js
└─ tailwind.config.js
```

## 3 Technology Stack & Tools

### 3.1 Frontend (The Client)

- **React.js**: Component-based UI library.
- **Vite**: Next-generation frontend build tool.
- **Tailwind CSS**: For rapid, utility-first styling.
- **WebSocket API**: Native browser API for real-time communication.

### 3.2 Middleware (The Bridge)

- **Node.js**: Runtime environment.
- **ws**: A simple to use, blazing fast WebSocket client/server for Node.js.
- **net**: Native Node.js module for creating TCP stream-based servers and clients.

### 3.3 Backend (The Server)

- **Java (JDK 8+)**: Core programming language.
- **Java IO/Net**: For `ServerSocket`, `Socket`, and `BufferedReader`.
- **MongoDB Java Driver**: For database connectivity.

### 3.4 Database

- **MongoDB Atlas**: Cloud-hosted NoSQL database.
- **Collections**: `users` (auth), `box_[username]` (message history).

## 4 Implementation Details

### 4.1 Backend Implementation

1. **ConnectionListener.java**: This runs a `'while(true)'` loop listening on port 12345. For every incoming TCP connection from the Bridge, it spawns a new **ClientWorker** thread.

2. **ClientWorker.java**: This class implements **Runnable**. It handles the client lifecycle:

- Authenticates the user (Login/Register).
- Listens for incoming commands (e.g., `T0:alice:hello`).
- Parses protocol strings and invokes the **ClientManager**.

3. **DatabaseManager.java**: Implements the "Mailbox" architecture. Instead of one giant messages table, every user has their own collection (e.g., `box_alice`). Messages are written twice (Sender's Box and Receiver's Box) to allow efficient history loading.

## 4.2 Bridge Implementation

1. **websocketService.js**: Creates a WebSocket Server on port 8080. When a web client connects, it triggers the creation of a TCP socket.

2. **tcpService.js**: Establishes a raw TCP connection to the Java backend. It implements **buffering** to handle TCP stream fragmentation, ensuring messages like `USERS:A,B` are processed correctly even if they arrive in chunks.

## 4.3 Frontend Implementation

1. **useChatBridge.js**: A custom React Hook that manages the WebSocket connection. It handles protocol parsing (converting `AUTH_SUCCESS` or `DM:alice:hi` into state updates).

2. **ChatMinimal.jsx**: The main UI component. It features an **Optimistic UI**, meaning messages appear in the chat window immediately upon sending, without waiting for server confirmation, providing a snappy user experience.

## 5 UML Architecture Diagram

